

IsWorking

Sistema de Control de Jornada Laboral con Geolocalización

MEMORIA TÉCNICA

Proyecto Final de Ciclo Formativo

Repositorio: github.com/IsWorking

Estado MVP: **44/44 pruebas superadas — PRODUCCIÓN READY**

Mayo 2025

Índice

Índice.....	2
1. Descripción General del Proyecto.....	4
2. Objetivos del Proyecto.....	5
2.1 Objetivo Principal.....	5
2.2 Objetivos Técnicos Específicos.....	5
3. Análisis de Requisitos.....	6
3.1 Requisitos Funcionales (RF).....	6
3.2 Requisitos No Funcionales (RNF).....	6
4. Planificación y Gestión del Tiempo.....	8
4.1 Desglose de Tareas y Estimaciones (Backlog).....	8
5. Análisis Funcional y Flujos.....	9
5.1 Diagrama de Flujo (User Flow).....	9
5.2 Interfaz de Usuario (UI).....	11
6. Arquitectura Técnica.....	13
6.1 Visión General.....	13
6.2 Patrón Multicapa del Backend.....	13
6.3 Stack Tecnológico Detallado.....	14
7. Modelo de Datos.....	15
7.1 Esquema Relacional — PostgreSQL.....	15
7.2 Esquema Documental — MongoDB.....	16
8. Gestión del Repositorio — Estrategia GitFlow.....	17
8.1 Estructura de Ramas.....	17
8.2 Pull Requests Principales.....	18
9. Funcionalidades del MVP.....	19
9.1 Sistema de Autenticación.....	19
9.2 Panel del Empleado.....	19
9.3 Panel del Administrador.....	19
9.4 Panel del Superadministrador.....	19
9.5 Sistema de Logs y Auditoría.....	20
10. API REST — Especificación de Endpoints.....	21
10.1 Autenticación — /api/auth.....	21
10.2 Usuarios — /api/users.....	21
10.3 Empresas — /api/companies.....	21
10.4 Turnos y Horarios.....	22
10.5 Fichajes — /api/records.....	22
10.6 Logs — /api/logs.....	22
11. Seguridad y Control de Acceso.....	23
11.1 Modelo de Autenticación.....	23
11.2 Modelo de Autorización por Roles.....	23
11.3 Medidas de Seguridad Adicionales.....	23
12. Suite de Pruebas — Resultados del MVP.....	24
12.1 Metodología de Pruebas.....	24

12.2 Resultados por Módulo.....	24
13. Retos Técnicos y Soluciones.....	27
13.1 Sincronización en la Inicialización del Servidor.....	27
13.2 Gestión de Fichajes en Escenario Offline.....	27
13.3 Filtros de Seguridad por Roles en Logs.....	27
14. Roadmap — Próximas Implementaciones.....	28
14.1 Prioridad Alta.....	28
14.2 Prioridad Media.....	28
14.3 Prioridad Baja.....	28
15. Despliegue y Configuración.....	29
15.1 Infraestructura con Docker Compose.....	29
15.2 Credenciales de Prueba.....	29
16. Uso de la Inteligencia Artificial.....	30
16.1 Metodología de Uso.....	30
16.2 Casos de Error Detectados.....	30
17. Conclusión.....	31

1. Descripción General del Proyecto

IsWorking es una aplicación web de control de jornada laboral con geolocalización diseñada para empresas que necesitan gestionar el fichaje de sus empleados de forma precisa, segura y auditable. Soporta escenarios de trabajo presencial y remoto, integrando captura automática de coordenadas GPS en cada registro de tiempo.

El sistema está orientado a tres perfiles de usuario diferenciados: el empleado que ficha su jornada, el administrador de empresa que supervisa y planifica turnos, y el superadministrador de plataforma que gestiona el ecosistema global de empresas y usuarios.

Dato	Valor
Nombre del proyecto	IsWorking
Tipo de producto	Aplicación web SaaS — Control de jornada laboral
Stack principal	Node.js · Express · PostgreSQL · MongoDB · React 18 · Vite
Arquitectura	API REST multicapa + SPA frontend · Persistencia políglota
Estado del MVP	44 de 44 pruebas automatizadas superadas (100%)
Despliegue local	Docker Compose (backend + PostgreSQL + MongoDB)
Puerto backend	:3000 (Express)
Puerto frontend	:5173 (Vite dev server)

2. Objetivos del Proyecto

2.1 Objetivo Principal

Desarrollar una API REST robusta, escalable y segura que resuelva la problemática del control de presencia laboral, permitiendo registrar con precisión las entradas, pausas y salidas del personal, capturando de forma obligatoria y transparente su ubicación GPS mediante el dispositivo desde el que se realiza la acción.

2.2 Objetivos Técnicos Específicos

- Demostrar la correcta implementación de metodologías de desarrollo de software moderno mediante GitFlow adaptado y feature branches.
- Integrar sistemas de persistencia heterogéneos: PostgreSQL para datos transaccionales con garantías ACID y MongoDB para logs de alta velocidad con esquemas flexibles.
- Implementar un sistema de autenticación seguro basado en JWT con cookies HttpOnly, access tokens de corta vida y refresh tokens de larga duración.
- Diseñar un control de acceso granular por roles (superadmin / admin / employee) validado en cada endpoint mediante middleware dedicado.
- Garantizar la trazabilidad completa de todas las operaciones sensibles mediante auditoría en MongoDB.
- Producir un frontend reactivo integrado que consuma la API de forma transparente, incluyendo renovación automática de tokens.

3. Análisis de Requisitos

Para garantizar la viabilidad y coherencia del MVP, se establecieron los siguientes requisitos formales antes de comenzar el desarrollo. La separación entre funcionales y no funcionales permite auditar con claridad qué hace el sistema y bajo qué restricciones técnicas opera.

3.1 Requisitos Funcionales (RF)

ID	Nombre	Descripción
RF-01	Autenticación y RBAC	El sistema diferenciará entre Superadmin, Admin y Empleado, con acceso a funcionalidades estrictamente diferentes según el rol asignado.
RF-02	Registro de fichajes con GPS	El empleado puede registrar entradas, pausas y salidas. En cada acción se capturan automáticamente las coordenadas GPS del dispositivo.
RF-03	Validación de secuencia de fichaje	El sistema impide acciones fuera de orden: no se puede iniciar pausa sin entrada previa, ni registrar salida sin haber cerrado la pausa.
RF-04	Gestión de turnos y horarios	El administrador crea plantillas de turno y las asigna a empleados con ciclo de vida: pending → confirmed → completed.
RF-05	Panel de administración de empresa	El administrador visualiza y gestiona todos los fichajes de su empresa con filtros por empleado, tipo, modalidad y fechas.
RF-06	Sistema de auditoría completo	Todas las operaciones sensibles quedan trazadas en MongoDB con actor, timestamp y payload completo.
RF-07	Sincronización de fichajes offline	El sistema almacena fichajes temporalmente sin conexión y los sincroniza al recuperarla, sin pérdida de datos ni duplicados.
RF-08	Gestión global de empresas	El superadmin crea, activa y desactiva empresas en operaciones atómicas que afectan automáticamente a todos sus usuarios.

3.2 Requisitos No Funcionales (RNF)

ID	Nombre	Descripción
RNF-01	Seguridad de autenticación	Rutas protegidas con JWT en cookie HttpOnly. Contraseñas hasheadas con bcrypt. Access token caduca en 15 minutos.
RNF-02	Aislamiento por empresa	Un administrador no puede acceder a datos de otra empresa. Validado en capa de servicio del backend, no solo en frontend.
RNF-03	Disponibilidad garantizada	El servidor no expone endpoints hasta que PostgreSQL y MongoDB confirman conexión exitosa.
RNF-04	Integridad referencial	El modelo relacional PostgreSQL usa Foreign Keys para evitar registros huérfanos al eliminar o desactivar entidades.

ID	Nombre	Descripción
RNF-05	Geolocalización irrenunciable	El formulario de fichaje queda bloqueado si el empleado revoca los permisos GPS. No es posible fichar sin coordenadas.
RNF-06	Escalabilidad de auditoría	La persistencia políglota permite escalar MongoDB de forma independiente a PostgreSQL sin impactar operaciones transaccionales.
RNF-07	Reproducibilidad del entorno	El entorno completo se levanta con un único comando (docker compose up --build) en cualquier máquina.

4. Planificación y Gestión del Tiempo

Para garantizar la entrega del MVP de forma ordenada, se realizó un desglose de tareas y una estimación de esfuerzo en horas efectivas antes de comenzar el desarrollo. Se aplicó un factor de seguridad del 20% sobre las estimaciones para cubrir bloqueos imprevistos y deuda técnica.

4.1 Desglose de Tareas y Estimaciones (Backlog)

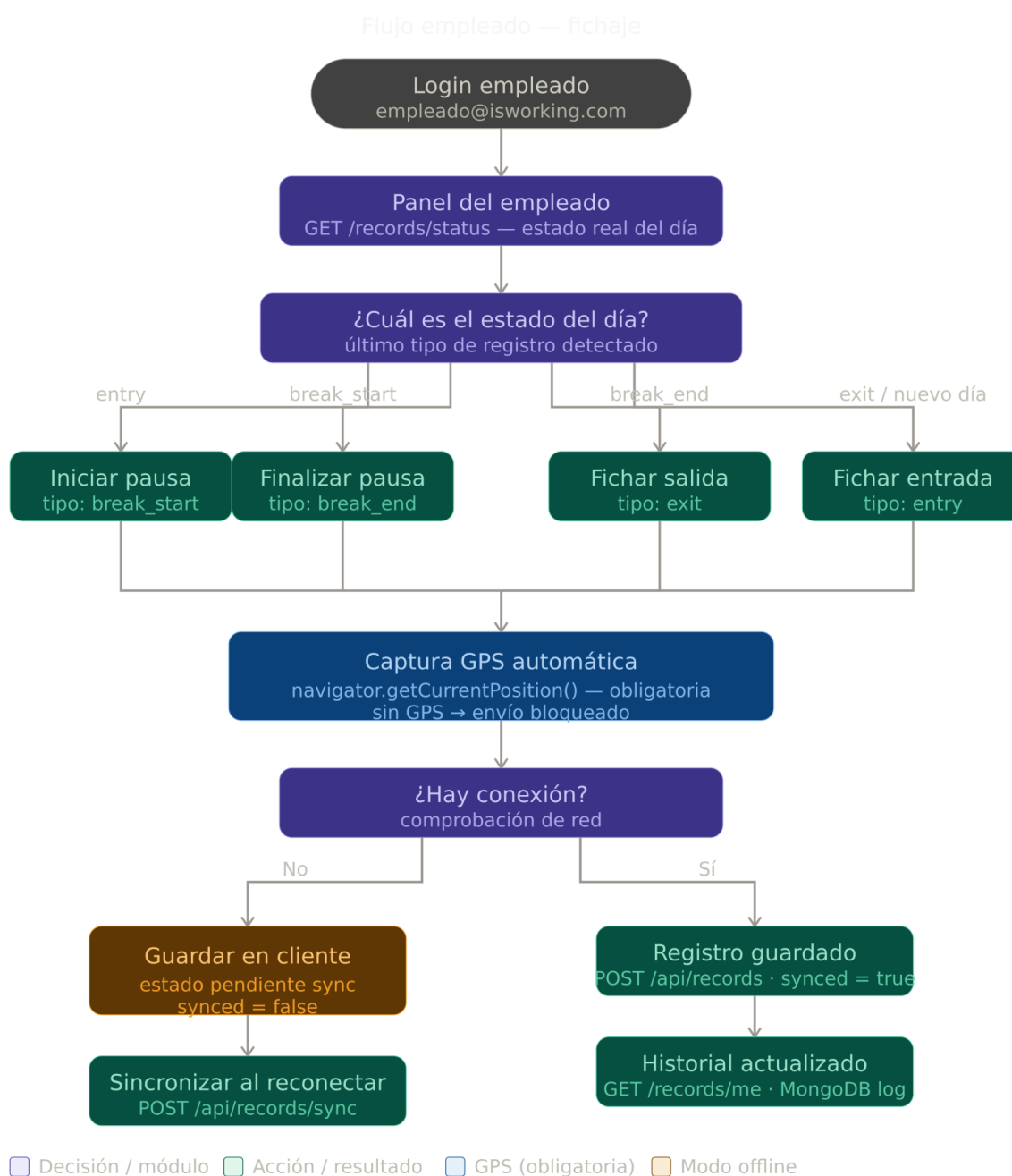
Fase	Tarea	Descripción técnica	Est.	Dependencias
Diseño	Modelo de datos	Diagrama E-R, entidades y relaciones PG + MongoDB. Contrato de API.	8h	—
Backend	Setup inicial	Express, CORS, Docker Compose, conexión dual PostgreSQL + MongoDB, .env.	6h	Diseño
Backend	Auth System	JWT, cookies HttpOnly, bcrypt, middleware protect.js, isAdmin.js.	12h	Setup
Backend	Core — Fichajes	Controladores, servicios y validación de secuencia de TimeRecord. GPS.	16h	Auth
Backend	Gestión de turnos	ShiftTemplates, Schedules, ciclo pending → confirmed → completed.	10h	Core
Backend	Gestión empresas	CRUD Superadmin. Operación atómica empresa + admin. Activación/desactivación.	8h	Auth
Backend	Logs y Auditoría	Colecciones logauths, logrecords, logadmins. Filtros por rol en servicios.	8h	Core
Backend	Sync Offline	Endpoint POST /api/records/sync. Lógica de deduplicación temporal.	6h	Core
Frontend	UI Base	React 18, React Router v7, design system --iw-*, layout y componentes base.	10h	Diseño
Frontend	Integración API	Axios, interceptor de refresh automático en 401, gestión de estados.	14h	Auth+EP
Frontend	Panel Empleado	Interfaz de fichaje, bloqueo GPS, reloj en tiempo real, historial filtrable.	12h	Integración
Frontend	Panel Admin	Gestión de empleados, turnos, horarios y registros con filtros.	12h	Integración
Frontend	Panel Superadmin	Gestión global de empresas y usuarios con activación/desactivación masiva.	8h	Integración
Cierre	QA & Pruebas	Batería de 44 pruebas de integración automatizadas (test_mvp.sh).	10h	Todo
Docs	Memoria técnica	Redacción de documentación, diagramas y limpieza del repositorio.	6h	—

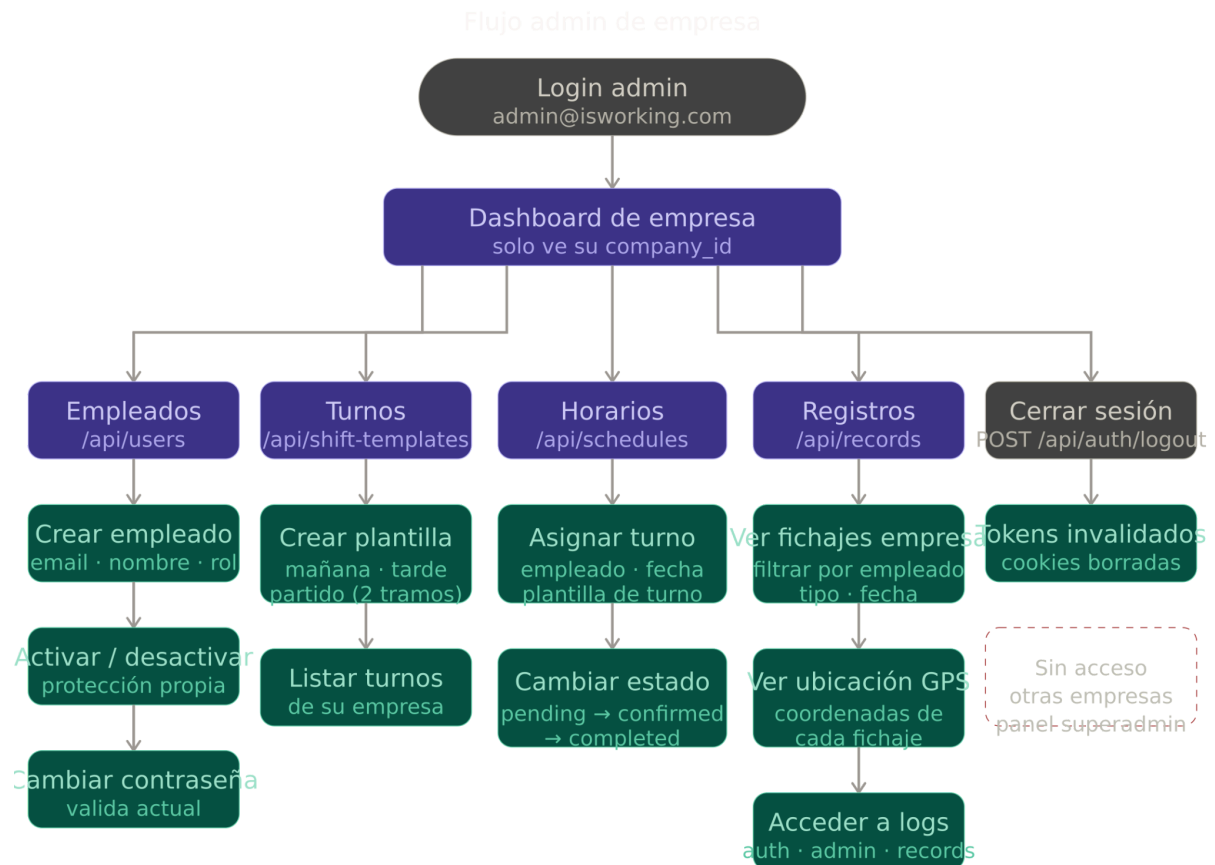
5. Análisis Funcional y Flujos

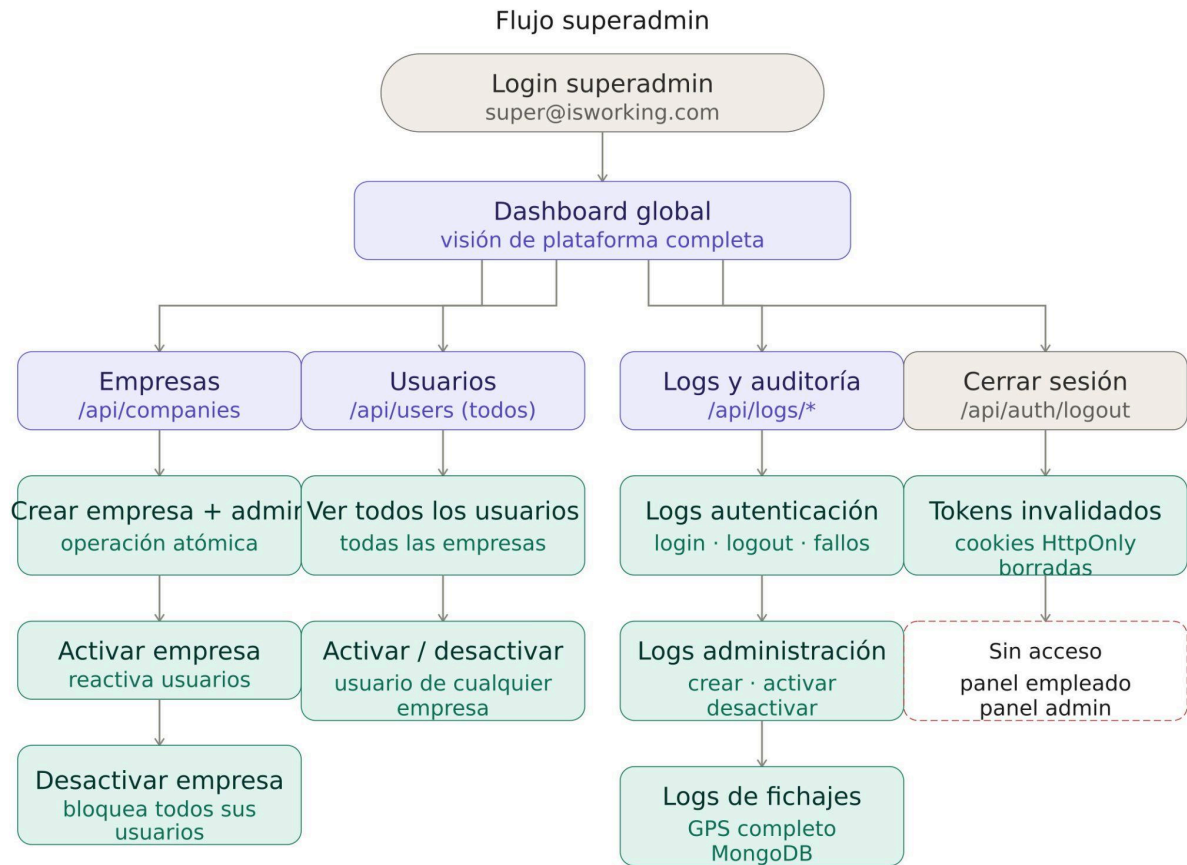
El sistema se articula en torno a tres actores principales con privilegios diferenciados: Empleado, Administrador y Superadministrador. A continuación se detalla el flujo principal de uso de la aplicación y las vistas de interfaz resultantes.

5.1 Diagrama de Flujo (User Flow)

El flujo principal de fichaje representa el recorrido del empleado, admin y superadmin desde el login hasta el cierre de jornada.







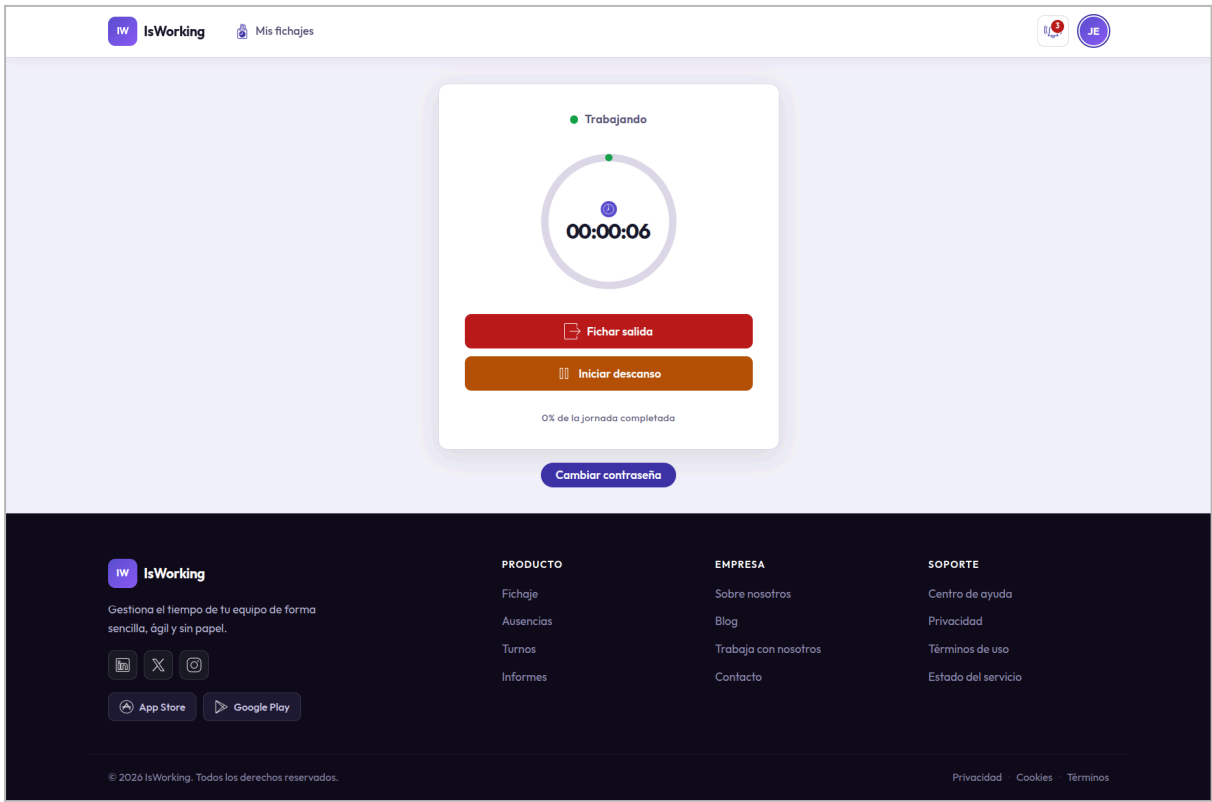
Módulo
 Acción
 Sesión
 Sin acceso

Los estados posibles de la jornada siguen la secuencia estricta que el backend valida en cada fichaje:

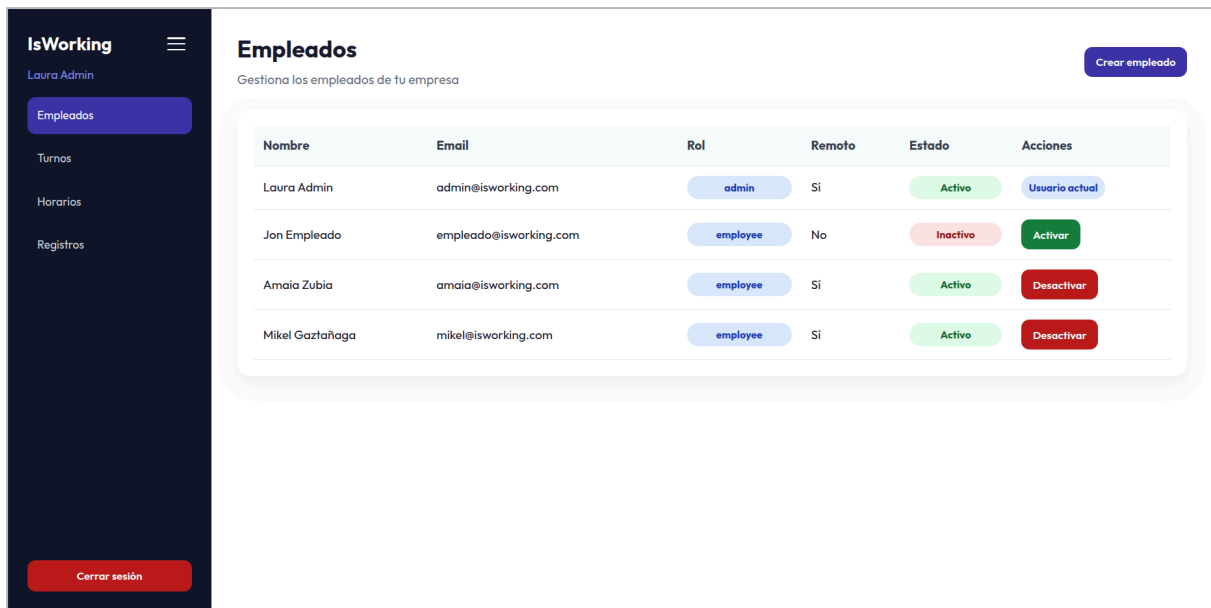
Estado actual	Acción disponible	Estado resultante	Descripción
Sin inicio	Fichar entrada	En jornada	Inicio de la jornada laboral. Captura GPS obligatoria.
En jornada	Iniciar pausa	En pausa	Interrupción de la jornada. Captura GPS obligatoria.
En pausa	Finalizar pausa	En jornada	Reanudación de la jornada. Captura GPS obligatoria.
En jornada	Fichar salida	Jornada cerrada	Fin de la jornada. Solo si no hay pausa activa.
Jornada cerrada	(ninguna)	—	Día completado. No se permiten más fichajes.

5.2 Interfaz de Usuario (UI)

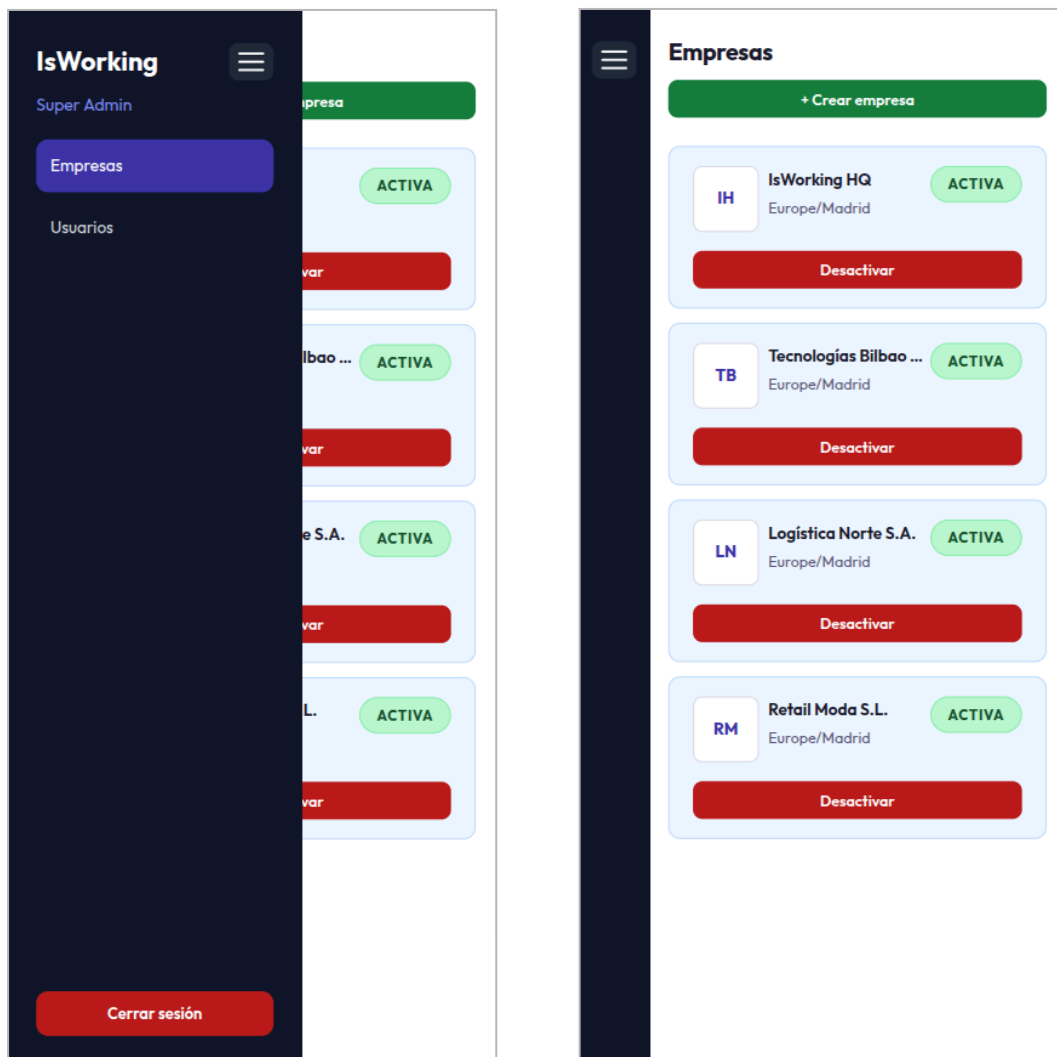
Se incluyen capturas del producto final para validar el cumplimiento de los requisitos de diseño y verificar la responsividad de las interfaces:



Panel del empleado en escritorio.



Panel de administrador en escritorio.



Panel de superadmin en móvil.

6. Arquitectura Técnica

6.1 Visión General

El ecosistema de IsWorking sigue una arquitectura de capas desacopladas con persistencia políglota. El frontend actúa como SPA que se comunica exclusivamente con el backend a través de la API REST. El backend centraliza toda la lógica de negocio y orquesta la escritura en dos sistemas de base de datos especializados.

FRONTEND — React 18 + Vite

SPA · puerto :5173 · proxy /api → :3000

BACKEND — Node.js + Express

API Gateway + Lógica de negocio · puerto :3000

Ruta → Middleware → Controlador → Servicio → Modelo

PostgreSQL 16 Datos transaccionales · Sequelize ORM · ACID

MongoDB 7 Logs y auditoría · Mongoose · alta velocidad

6.2 Patrón Multicapa del Backend

Cada petición HTTP transita un pipeline fijo que garantiza separación de responsabilidades y facilita el testing unitario de cada capa de forma independiente:

Capa	Responsabilidad	Ejemplos
Routes	Definición de endpoints y método HTTP	/api/records, /api/auth/login
Middlewares	Autenticación, autorización por rol, captura de errores	protect.js, isAdmin.js, errorHandler.js
Controllers	Recepción de req/res, validación de entrada, delegación al service	recordController.js, authController.js
Services	Lógica de negocio pura, orquestación de modelos	recordService.js, scheduleService.js
Models (PG)	Entidades Sequelize y sus asociaciones	Company, User, TimeRecord, Schedule
Models (Mongo)	Esquemas Mongoose para auditoría	LogAuth, LogRecord, LogAdmin

6.3 Stack Tecnológico Detallado

Backend

Tecnología	Versión	Rol
Node.js	LTS	Runtime de servidor
Express	4.x	Framework HTTP y router
PostgreSQL	16	Base de datos relacional principal
Sequelize	6.x	ORM para PostgreSQL
MongoDB	7	Base de datos documental para logs
Mongoose	7.x	ODM para MongoDB
JSON Web Tokens	Access 15 min / Refresh 7d	Autenticación stateless
bcrypt	Última estable	Hashing seguro de contraseñas
Docker Compose	v2	Orquestación de infraestructura local

Frontend

Tecnología	Versión	Rol
React	18	Librería de UI reactiva
Vite	Última	Bundler y servidor de desarrollo
react-router-dom	v7	Enrutado de SPA
Axios	Última	Cliente HTTP con interceptor de refresh automático
CSS Variables	—	Sistema --iw-* Design system propio

7. Modelo de Datos

7.1 Esquema Relacional — PostgreSQL

La capa transaccional gestiona las entidades de negocio con integridad referencial completa. Las cinco tablas del esquema garantizan consistencia ACID para todas las operaciones críticas:

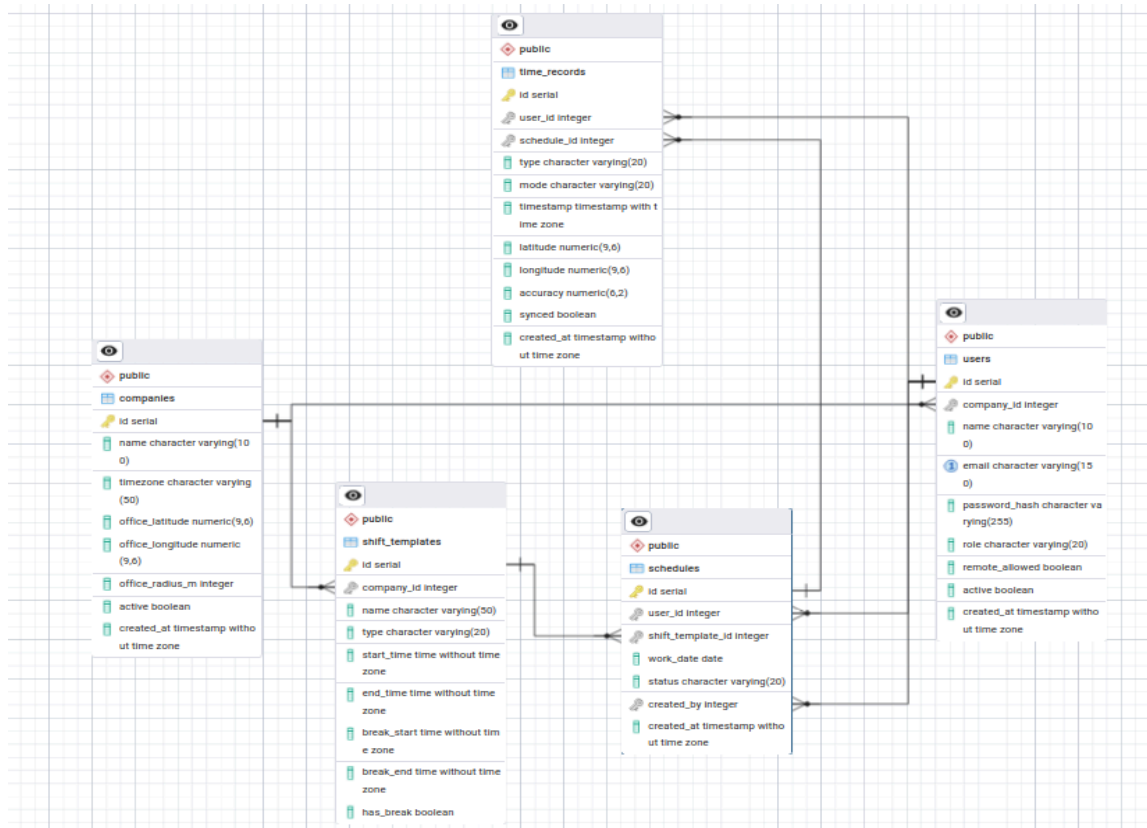


Tabla	Descripción	Relaciones clave
companies	Empresas registradas en la plataforma. Almacena nombre, datos de contacto, coordenadas de oficina y radio de geofencing.	Parent de users
users	Usuarios del sistema con rol diferenciado (superadmin / admin / employee). Incluye estado activo/inactivo y pertenencia a empresa.	Child de companies · Parent de schedules y time_records
shift_templates	Plantillas de turno reutilizables (mañana, tarde, partido). Definen franjas horarias con soporte de doble tramo.	Parent de schedules
schedules	Asignación de un turno a un empleado para una fecha concreta. Estado de ciclo de vida: pending / confirmed / completed.	Child de users y shift_templates

Tabla	Descripción	Relaciones clave
time_records	Fichajes individuales con timestamp, tipo de acción, modalidad de trabajo y coordenadas GPS.	Child de users

Relaciones entre entidades

- Company (1) → (N) User: Una empresa puede tener múltiples empleados y administradores. La desactivación bloquea automáticamente a todos sus usuarios.
- User (1) → (N) Schedule: Un empleado tiene un histórico de planificaciones de turnos, permitiendo auditar cualquier periodo.
- User (1) → (N) TimeRecord: Cada fichaje queda vinculado al usuario que lo generó, con su timestamp y geolocalización.
- ShiftTemplate (1) → (N) Schedule: Una plantilla de turno puede reutilizarse en múltiples asignaciones de calendario.

7.2 Esquema Documental — MongoDB

La capa de auditoría utiliza tres colecciones optimizadas para escrituras de alta frecuencia. La naturaleza documental de MongoDB permite capturar payloads variables sin penalizar el rendimiento:

Colección	Contenido	Eventos registrados
logauths	Historial completo de sesiones de usuario.	Login exitoso, login fallido, logout, refresh de token, credenciales bloqueadas
logrecords	Registro de cada fichaje con coordenadas GPS completas.	Entrada, inicio de pausa, fin de pausa, salida, sincronización offline
logadmins	Trazabilidad de acciones administrativas.	Crear usuario, activar/desactivar usuario, crear/activar/desactivar empresa, asignar horario

8. Gestión del Repositorio — Estrategia GitFlow

8.1 Estructura de Ramas

El desarrollo se organizó bajo GitFlow adaptado, garantizando que main nunca recibiera commits directos. Cada funcionalidad fue aislada en su propia feature branch y unificada únicamente tras pasar pruebas locales satisfactorias.

```
isworking/
├── backend/
│   ├── src/
│   │   ├── controllers/    # Recibe req/res, delega al service
│   │   ├── services/      # Lógica de negocio
│   │   ├── routes/        # Definición de endpoints
│   │   ├── middlewares/   # protect, isAdmin, errorHandler
│   │   ├── models/
│   │   │   ├── postgres/  # Company, User, ShiftTemplate, Schedule, TimeRecord
│   │   │   └── mongo/     # LogAuth, LogRecord, LogAdmin
│   │   └── config/        # postgres.js, mongo.js
│   ├── Dockerfile
│   ├── docker-compose.yml
│   └── .env.example
├── frontend/
│   ├── src/
│   │   ├── pages/         # Dashboard, historial, admin, superadmin
│   │   ├── components/    # Clock, fichaje, tablas
│   │   └── services/       # Llamadas a la API
│   └── vite.config.js
└── DOCS/
    ├── scripts/
    └── test_mvp.sh        # Batería de 44 pruebas automatizadas
```

Rama	Tipo	Propósito
main	Producción	Estado estable, auditado y completamente funcional. Punto de despliegue.
dev	Integración backend	Recibe el código verificado de features antes de pasar a main.
frontend	Integración frontend	Rama de integración paralela para la capa de vistas React.
feat/time	Feature	Núcleo del sistema de fichajes: controladores, rutas y servicios de TimeRecord.
feat/auth	Feature	Implementación completa de seguridad: JWT, cookies, middleware de roles.
feat/schedule	Feature	Gestión y persistencia de calendarios laborales.
feat/shift	Feature	Lógica de plantillas de turno (mañana / tarde / partido).
feat/employee	Feature	Ciclo de vida de empleados y asignación de roles.

Rama	Tipo	Propósito
feat/user	Feature	Gestión de usuarios a nivel de plataforma (superadmin).
feat/pruebas	QA	Entorno de auditoría interna. Fusión temporal para verificar coherencia de asociaciones.
feat/monologs	QA	Gestión de logs MongoDB y verificación de comportamiento de auditoría.
feature/postgres-migratio n	Feature	Migración y separación del modelo relacional. Introduce src/models/postgres/ e init.sql.
feature/logs-mongodb	Feature	Desarrollo de la capa de persistencia de auditorías de rendimiento y seguridad.

8.2 Pull Requests Principales

PR.1 — Cimentación de la API

Unificación del indexador global de Express con el setup inicial de middlewares para CORS, parseo de cookies y manejo centralizado de errores. Establece la estructura base del servidor sobre la que se apoyan todas las features posteriores.

PR.2 — Implementación de Persistencia Dual

Migración drástica que segmenta el modelo de datos inicial en las carpetas especializadas /postgres y /mongo. Introduce las asociaciones de Sequelize centralizadas en associations.js y la conexión diferenciada a MongoDB mediante Mongoose. Resuelve el problema de sincronización en la inicialización.

PR.3 — Blindaje y Sanitización

Adición de la capa completa de middlewares de control de acceso por roles. Implementa los guards protect.js, isAdmin.js y el capturador global de errores que garantiza que ninguna excepción no controlada derribe el servidor.

9. Funcionalidades del MVP

9.1 Sistema de Autenticación

- Login con email y contraseña mediante bcrypt. Generación de access token (15 min) y refresh token (7 días) como cookies HttpOnly.
- Refresh automático transparente: el interceptor de Axios detecta respuestas 401 y renueva la sesión sin interrumpir la experiencia del usuario.
- Logout con invalidación completa de sesión en servidor y cliente.
- Cambio de contraseña con validación de contraseña actual antes de permitir el cambio.
- Bloqueo de sesión para usuarios o empresas desactivadas: acceso denegado en el primer middleware.

9.2 Panel del Empleado

- Interfaz de fichaje con cuatro acciones: entrada, inicio de pausa, fin de pausa y salida.
- Validación estricta de secuencia: el sistema impide acciones fuera de orden (p.ej. fichar salida sin entrada previa).
- Geolocalización automática e irrenunciable en cada acción de fichaje. El formulario queda bloqueado si los permisos GPS son revocados.
- Recuperación de estado real al recargar: el reloj arranca con el estado del día en curso, garantizando coherencia aunque el usuario haya cerrado el navegador.
- Historial de fichajes filtrable por mes con fecha, hora, tipo de acción, modalidad de trabajo y coordenadas GPS capturadas.

9.3 Panel del Administrador

- Gestión de empleados: creación, activación y desactivación con protección contra autodesactivación accidental.
- Plantillas de turno: creación de turnos de mañana, tarde y partido (con dos tramos horarios independientes).
- Asignación de horarios: vinculación de plantilla de turno a empleado por fecha con gestión del ciclo de estados (pending → confirmed → completed).
- Registros: visualización de todos los fichajes de la empresa con filtros por empleado, tipo de acción, modalidad y rango de fechas.
- Aislamiento de datos: el administrador únicamente accede a la información de su empresa, validado en el backend.

9.4 Panel del Superadministrador

- Gestión de empresas: creación de empresa con su administrador incluido en una sola operación atómica.
- Activación y desactivación de empresas: la desactivación bloquea automáticamente a todos los usuarios asociados sin necesidad de acción individual.

- Visión global de usuarios: acceso a todos los usuarios de todas las empresas con capacidad de activación/desactivación.

9.5 Sistema de Logs y Auditoría

- Logs de autenticación: cada evento de login, logout, refresh o intento fallido queda registrado con timestamp, IP y rol de usuario.
- Logs de administración: todas las acciones administrativas (crear entidad, cambiar estado) quedan trazadas con el actor que las ejecutó.
- Logs de fichajes: cada TimeRecord genera un documento en MongoDB con las coordenadas GPS completas.
- Control de acceso a logs por rol: los empleados tienen bloqueado el acceso a cualquier ruta de auditoría.

10. API REST — Especificación de Endpoints

10.1 Autenticación — /api/auth

Método	Endpoint	Rol mínimo	Descripción
POST	/api/auth/login	Público	Autenticación con email y contraseña. Devuelve cookies HttpOnly.
POST	/api/auth/logout	Autenticado	Cierre de sesión e invalidación de tokens.
POST	/api/auth/refresh	Autenticado	Renovación del access token usando el refresh token de la cookie.

10.2 Usuarios — /api/users

Método	Endpoint	Rol mínimo	Descripción
GET	/api/users	Admin	Lista empleados de la empresa del admin. Superadmin ve todos.
POST	/api/users	Admin	Crea un nuevo empleado asociado a la empresa del admin.
PATCH	/api/users/:id	Admin	Actualiza datos del empleado (nombre, email, turno...).
PATCH	/api/users/:id/status	Admin	Activa o desactiva un empleado. Protección contra autodesactivación.
PATCH	/api/users/me/password	Empleado	Cambia la contraseña propia validando la contraseña actual.

10.3 Empresas — /api/companies

Método	Endpoint	Rol mínimo	Descripción
GET	/api/companies	Superadmin	Lista todas las empresas registradas en la plataforma.
POST	/api/companies	Superadmin	Crea empresa junto con su usuario administrador en operación atómica.
PATCH	/api/companies/:id/activate	Superadmin	Activa una empresa y reactiva sus usuarios.
PATCH	/api/companies/:id/deactivate	Superadmin	Desactiva empresa y bloquea a todos sus usuarios automáticamente.

10.4 Turnos y Horarios

Método	Endpoint	Rol mínimo	Descripción
GET	/api/shift-templates	Admin	Lista las plantillas de turno disponibles en la empresa.
POST	/api/shift-templates	Admin	Crea una nueva plantilla de turno (mañana / tarde / partido).
GET	/api/schedules	Admin	Lista los horarios asignados. Empleado solo ve los suyos.
POST	/api/schedules	Admin	Asigna un turno a un empleado para una fecha concreta.
PATCH	/api/schedules/:id/status	Admin	Cambia el estado del horario (pending → confirmed → completed).

10.5 Fichajes — /api/records

Método	Endpoint	Rol mínimo	Descripción
GET	/api/records	Admin	Registros de la empresa con filtros. Empleado recibe solo los suyos.
GET	/api/records/me	Empleado	Historial de fichajes del empleado autenticado.
GET	/api/records/status	Empleado	Estado actual del día: tipo del último fichaje y acción disponible.
POST	/api/records	Empleado	Crea un nuevo fichaje con geolocalización (entrada / pausa / salida).
POST	/api/records/sync	Empleado	Sincroniza los fichajes generados en modo offline al recuperar conexión.

10.6 Logs — /api/logs

Método	Endpoint	Rol mínimo	Descripción
GET	/api/logs/auth	Admin	Historial de eventos de autenticación de la empresa.
GET	/api/logs/admin	Admin	Trazabilidad de acciones administrativas realizadas.
GET	/api/logs/records	Admin	Log de fichajes con coordenadas GPS completas.

11. Seguridad y Control de Acceso

11.1 Modelo de Autenticación

El sistema implementa autenticación stateless mediante JWT almacenados en cookies HttpOnly, eliminando la superficie de ataque XSS sobre los tokens. El ciclo de vida de la sesión se controla mediante dos tokens con vidas útiles diferenciadas:

Token	Vida útil	Almacenamiento	Propósito
Access Token	15 minutos	Cookie HttpOnly (Secure)	Autorización de requests a endpoints protegidos
Refresh Token	7 días	Cookie HttpOnly (Secure)	Renovación transparente del access token caducado

11.2 Modelo de Autorización por Roles

Capacidad	Empleado	Admin	Superadmin
Fichar entrada/pausa/salida	✓	✓	✓
Ver historial propio	✓	✓	✓
Ver registros de empresa	—	✓	✓
Gestionar empleados	—	✓	✓
Gestionar turnos y horarios	—	✓	✓
Acceder a logs de auditoría	—	✓	✓
Gestionar empresas	—	—	✓
Ver todos los usuarios	—	—	✓

11.3 Medidas de Seguridad Adicionales

- Hashing de contraseñas con bcrypt (salt rounds configurables). Las contraseñas nunca se almacenan en texto plano.
- Bloqueo automático de cuentas: usuarios y empresas desactivadas reciben 401 en el primer middleware, antes de ejecutar cualquier lógica.
- Aislamiento por empresa: los administradores solo pueden acceder a datos de su empresa. Validado en capa de servicio, no solo en frontend.
- Gestión de credenciales por entornos: el archivo .env.example centraliza la configuración sin exponer credenciales en el repositorio.
- Políticas restrictivas de CORS: solo se aceptan peticiones desde orígenes autorizados.
- Control de errores centralizado: el errorHandler global captura excepciones no controladas sin exponer stack traces al cliente.

12. Suite de Pruebas — Resultados del MVP

12.1 Metodología de Pruebas

El proyecto incluye una batería automatizada de pruebas de integración (test_mvp.sh) que valida el comportamiento end-to-end de todos los módulos de la API mediante peticiones HTTP reales contra el servidor en ejecución. Las pruebas cubren el 100% de los flujos críticos del MVP.

Total pruebas	Correctas	Tasa de éxito
44	44	100%

12.2 Resultados por Módulo

1. Health Check	Resultado
Backend responde en /health	✓ PASS

2. Autenticación	Resultado
Login superadmin	✓ PASS
Login admin	✓ PASS
Login empleado	✓ PASS
Credenciales incorrectas bloqueadas	✓ PASS
Refresh token funciona	✓ PASS

3. Usuarios	Resultado
Admin lista empleados de su empresa	✓ PASS
Superadmin lista todos los usuarios	✓ PASS
Crear empleado	✓ PASS
Desactivar empleado	✓ PASS
Empleado desactivado bloqueado al hacer login	✓ PASS
Reactivar empleado	✓ PASS
Protección autodesactivación del admin	✓ PASS
Cambiar contraseña de empleado	✓ PASS
Contraseña actual incorrecta bloqueada	✓ PASS

4. Empresas (Superadmin)	Resultado
Superadmin lista empresas	✓ PASS
Crear empresa con admin	✓ PASS
Admin creado con empresa puede loguearse	✓ PASS
Desactivar empresa	✓ PASS
Admin de empresa desactivada bloqueado	✓ PASS
Activar empresa	✓ PASS

5. Turnos	Resultado
Crear plantilla de turno	✓ PASS
Listar plantillas de turno	✓ PASS

6. Horarios	Resultado
Asignar horario a empleado	✓ PASS
Cambiar estado de horario a confirmed	✓ PASS
Admin lista horarios de la empresa	✓ PASS
Empleado lista únicamente sus horarios	✓ PASS

7. Fichajes (Empleado)	Resultado
GET /records/status devuelve estado del día	✓ PASS
Fichar salida (completar día anterior abierto)	✓ PASS
Fichar entrada	✓ PASS
Iniciar pausa	✓ PASS
Finalizar pausa	✓ PASS
Fichar salida	✓ PASS
Secuencia de fichaje validada correctamente	✓ PASS
GET /records/me devuelve historial del empleado	✓ PASS

8. Registros (Admin)	Resultado
Admin ve registros de su empresa	✓ PASS
Empleado accede a registros filtrados por rol en backend	✓ PASS

9. Logs (MongoDB)	Resultado
Logs de autenticación accesibles para admin	✓ PASS

9. Logs (MongoDB)	Resultado
Logs de administración accesibles para admin	✓ PASS
Logs de fichajes accesibles para admin	✓ PASS
Empleado bloqueado de todas las rutas de logs	✓ PASS

10. Logout	Resultado
Logout admin	✓ PASS
Logout empleado	✓ PASS
Logout superadmin	✓ PASS

13. Retos Técnicos y Soluciones

13.1 Sincronización en la Inicialización del Servidor

Problema: las consultas fallaban al arrancar el servidor porque las asociaciones de Sequelize no estaban construidas antes de que Express comenzara a escuchar peticiones. Los endpoints respondían con errores de relaciones indefinidas.

Solución: centralización de todas las dependencias entre modelos en `src/models/postgres/associations.js`, con ejecución síncrona garantizada en `index.js` antes de la apertura del puerto. Ningún endpoint queda expuesto hasta que ambas bases de datos confirman conexión exitosa.

13.2 Gestión de Fichajes en Escenario Offline

Problema: identificado durante el desarrollo, existe el riesgo de pérdida de datos si un empleado pierde la conexión a internet en el momento de fichar. El fichaje se ejecuta en cliente pero nunca llega al servidor.

Solución: implementación de un endpoint específico `POST /api/records/sync` y su interceptor en los controladores. El cliente almacena temporalmente el estado del fichaje cuando detecta ausencia de conectividad y lo sincroniza con el servidor en cuanto la conexión se restaura, sin pérdida de datos ni duplicados.

```
// Lógica simplificada de sincronización offline (cliente)
async function syncOfflineRecords() {
  const pending = JSON.parse(localStorage.getItem('offline_records') || '[]');
  if (!pending.length) return;
  await axios.post('/api/records/sync', { records: pending });
  localStorage.removeItem('offline_records');
}
```

13.3 Filtros de Seguridad por Roles en Logs

Problema: el volcado inicial de auditoría exponía datos sensibles de todos los usuarios al consultar las rutas de logs, independientemente del rol del solicitante.

Solución: modificación de los servicios de logs para extraer y validar el rol del usuario directamente desde la cookie de sesión antes de construir la query. Un empleado recibe 403 inmediatamente. Un administrador recibe únicamente los logs de su empresa. Solo el superadministrador accede al volcado global.

14. Roadmap — Próximas Implementaciones

14.1 Prioridad Alta

Funcionalidad	Descripción
Edición de fichajes por admin	Correcciones manuales de registros erróneos con trazabilidad del cambio.
Alertas de jornada abierta	Notificación automática cuando un empleado lleva más de X horas sin fichar salida.
Validación de geofencing en backend	Actualmente la ubicación se guarda pero no se valida contra el radio configurado en la empresa. Pendiente activar la comprobación real.
Gestión de ausencias y vacaciones	Módulo de solicitud y aprobación de ausencias integrado con el calendario de turnos.
Dashboard de KPIs para admin	Panel con empleados fichados en tiempo real, ausencias del día y horas totales de la semana.

14.2 Prioridad Media

Funcionalidad	Descripción
Exportación a CSV/Excel	Descarga de registros de fichajes por periodo y empleado.
Paginación	Tablas de registros y horarios con paginación server-side para grandes volúmenes.
Edición de empresa	Modificación de coordenadas de oficina, radio de geofencing y zona horaria.
Recuperación de contraseña	Flujo de reset por email con token de un solo uso.

14.3 Prioridad Baja

Funcionalidad	Descripción
App móvil nativa	Cliente iOS/Android con modo offline y sincronización vía POST /api/records/sync (ya implementado en backend).
Soporte para múltiples pausas	Gestión de más de una pausa por jornada con cálculo acumulado de tiempo muerto.
Informes mensuales automáticos	Generación y envío de resumen de horas por empleado al cierre de cada mes.
Integración con calendarios externos	Sincronización bidireccional con Google Calendar y Outlook.
SSO / Login con Google	Autenticación federada mediante OAuth 2.0.

15. Despliegue y Configuración

15.1 Infraestructura con Docker Compose

El entorno de desarrollo completo se levanta con un único comando. Docker Compose orquesta tres servicios: el servidor Express, PostgreSQL y MongoDB. Las variables de entorno se gestionan mediante .env (ignorado por Git) partiendo de la plantilla .env.example incluida en el repositorio.

```
# Levantar backend completo (Express + PostgreSQL + MongoDB)
cd backend
docker compose up --build

# Levantar frontend
cd frontend
npm install && npm run dev

# Ejecutar batería de pruebas MVP
chmod +x test_mvp.sh && ./test_mvp.sh
```

15.2 Credenciales de Prueba

Rol	Email	Contraseña	Permisos
Superadmin	super@isworking.com	isworking123	Acceso total a plataforma
Admin	admin@isworking.com	isworking123	Gestión de empresa propia
Empleado	empleado@isworking.com	isworking123	Fichaje y consulta propia

16. Uso de la Inteligencia Artificial

El uso de IA en el desarrollo de software plantea una responsabilidad clara: el código generado debe ser comprendido por quien lo integra. A lo largo del proyecto se ha pedido sistemáticamente una explicación de cada bloque de código antes de aplicarlo, lo que ha convertido la IA en una herramienta de aprendizaje acelerado en lugar de un atajo que omite la comprensión.

La IA comete errores (como ocurrió con comentarios JSX mal colocados dentro de paréntesis, o imports pegados en archivos CSS), y detectar esos errores requiere conocimiento propio. Esa capacidad crítica es la que distingue el uso responsable del uso irreflexivo de estas herramientas.

16.1 Metodología de Uso

- Antes de integrar cualquier fragmento de código generado, se solicitó una explicación detallada de su funcionamiento, sus dependencias y sus posibles efectos secundarios.
- La IA se utilizó como par de programación acelerado, no como sustituto del razonamiento técnico propio. Cada decisión arquitectónica (persistencia políglota, middleware de roles, ciclo de tokens JWT) fue comprendida y validada de forma independiente.
- Los errores generados por la IA fueron identificados y corregidos gracias a la comprensión previa del stack tecnológico, demostrando que el conocimiento propio es condición necesaria para un uso responsable.
- La batería de 44 pruebas automatizadas sirvió también como mecanismo de validación del código asistido por IA, garantizando que cada bloque integrado cumpliera el comportamiento esperado de forma objetiva.

16.2 Casos de Error Detectados

Error generado por IA	Contexto	Cómo se detectó y corrigió
Comentarios JSX mal colocados dentro de paréntesis de retorno	Componentes React del panel de fichaje	El compilador de Vite lanzó un error de parsing. Conocimiento de la sintaxis JSX permitió identificar la causa y mover los comentarios fuera del bloque de retorno.
Imports pegados directamente en archivos CSS	Integración del design system --iw--*	El bundler ignoró los imports. Revisión del código generado y conocimiento de cómo Vite procesa los assets CSS permitió corregirlo.
Asociaciones de Sequelize definidas fuera del orden correcto de carga	Modelo de datos inicial de PostgreSQL	Los endpoints devolvían errores de relaciones indefinidas. Se reorganizaron las definiciones en associations.js con carga síncrona garantizada.

Esta capacidad crítica es la que distingue el uso responsable del uso irreflexivo de estas herramientas, y refleja el nivel de comprensión técnica alcanzado a lo largo del proyecto.

17. Conclusión

IsWorking representa un caso de éxito en el desarrollo ágil de un producto web completo en el contexto de un proyecto de ciclo formativo. El resultado final es un sistema de control de jornada laboral plenamente operativo, con una arquitectura limpia, un stack tecnológico moderno y un nivel de acabado técnico riguroso.

El 100% de las 44 pruebas automatizadas superadas certifica la solidez del MVP. El historial del repositorio demuestra el uso ejemplar de herramientas de control de versiones, donde el equipo trabajó en paralelo sobre componentes sensibles (autenticación, bases de datos duales, capturas de geolocalización) sin generar conflictos de código destructivos.

Pilar técnico	Evidencia
Arquitectura limpia	Separación estricta en capas: routes → middleware → controller → service → model. Sin lógica de negocio en controladores.
Persistencia políglota	PostgreSQL para integridad transaccional. MongoDB para auditoría de alta velocidad. Cada tecnología donde aporta más valor.
Seguridad robusta	JWT en cookies HttpOnly, roles diferenciados, aislamiento por empresa, logs de auditoría completos.
Trazabilidad del desarrollo	GitFlow adaptado, feature branches, PR documentadas, historial de commits atómicos y semánticos.
Calidad certificada	44/44 pruebas de integración automatizadas. MVP declarado completamente funcional.
Modernidad tecnológica	Node.js + React 18 + PostgreSQL 16 + MongoDB 7. Docker Compose para reproducibilidad del entorno.
IA como herramienta crítica	Uso responsable con comprensión activa del código generado, validación sistemática y capacidad de detectar y corregir errores.

El proyecto está en condiciones óptimas para ser presentado y defendido ante cualquier tribunal de evaluación, y sienta las bases técnicas para su evolución hacia un producto SaaS escalable.